

ORMs

COMPLETE LEARNING GUIDE

What You'll Learn

An Object-Relational Mapper lets application code work with database records as ordinary objects (or typed models) instead of writing raw SQL strings for every operation, and modern ORM's like Prisma, Sequelize, and TypeORM add schema definitions, migrations, and relationship handling on top of that core mapping. This guide covers what problem ORM's actually solve, how models and migrations version a schema over time, how relationships (one-to-many, many-to-many) are declared and queried, the eager-vs-lazy loading distinction that's central to avoiding performance problems, and — just as important — recognizing the specific situations where dropping down to raw SQL is the better choice than forcing an ORM to express something it isn't well suited for.

1. What Problem ORM's Solve

Without an ORM, every database interaction means writing a raw SQL string, manually mapping the returned rows into application objects, and hand-writing schema changes as SQL migration scripts. An ORM removes most of that repetition: you define a model once (its fields and types), and the ORM generates the corresponding SQL for common operations (create, read, update, delete, and filtered queries) through a typed, language-native API — which also gives you compile-time type checking on queries in a typed language like TypeScript, catching mistakes (a typo'd field name, a wrong type) before the query ever runs.

2. Defining Models and Schema

In Prisma, the schema is defined declaratively in a single `schema.prisma` file, describing each model's fields, types, and relationships; the ORM then generates a fully-typed client from that schema. Other ORM's (Sequelize, TypeORM) define models directly in code as classes or objects with decorators. Either way, the model definition becomes the single source of truth the ORM uses both to generate queries and to generate/validate migrations against the actual database schema.

TYPESCRIPT

```
// schema.prisma
model Student {
  id          String      @id @default(cuid())
  name        String
  email       String      @unique
  enrollments Enrollment[]
  createdAt   DateTime     @default(now())
}

model Enrollment {
  id          String      @id @default(cuid())
  studentId   String
  student     Student     @relation(fields: [studentId], references: [id])
  courseId    String
  status      String      @default("active")
}
```

3. Migrations and Schema Versioning

A migration is a versioned, ordered set of schema changes (create table, add column, add index) that lets a database schema evolve safely and reproducibly across every environment — local, staging, production — rather than manually running ad hoc ALTER TABLE statements that different environments might apply differently or forget entirely. Most ORMs can auto-generate a migration by diffing the current schema definition against the previous one, though reviewing the generated SQL before applying it to production is still essential, since destructive changes (dropping a column with data in it) deserve a human decision, not silent automation.

4. Querying Through the ORM

ORM query APIs translate method chains or query objects into SQL under the hood, typically supporting filtering, sorting, pagination, and field selection through a fluent, language-native syntax rather than string-built SQL. The productivity gain is real, but it comes with an important discipline: periodically checking the actual generated SQL (most ORMs expose a query-logging option) to confirm it's doing what you expect, especially for anything beyond a simple single-table lookup.

TYPESCRIPT

```
const activeStudents = await prisma.student.findMany({
  where: { enrollments: { some: { status: "active" } } },
  orderBy: { createdAt: "desc" },
  take: 20,
  select: { id: true, name: true, email: true },
});
```

5. Relationships: One-to-Many and Many-to-Many

A one-to-many relationship (one student, many enrollments) is declared with a foreign key on the 'many' side, exactly mirroring the underlying relational foreign key. A many-to-many relationship (students to courses, where a student takes many courses and a course has many students) requires a join table under the hood — some ORMs (Prisma) can manage this join table implicitly for the simple case, while more complex many-to-many relationships that need their own extra fields (like an enrollment date on the join itself) require modeling the join table as its own explicit model, exactly as you would in raw SQL.

6. Eager vs. Lazy Loading

Lazy loading fetches a related record only when it's actually accessed in code, which is convenient but is exactly what causes the N+1 query problem when that access happens inside a loop over many parent records. Eager loading fetches related data up front, in the same query (via a JOIN) or in one additional batched query, and is the standard fix once you know you'll need the related data for every item in a list. Every mainstream ORM supports eager loading explicitly (Prisma's `include`, Sequelize's `include`, TypeORM's `relations` option) — the discipline is remembering to reach for it whenever a list of parent records and their related data are both needed together.

7. When to Bypass the ORM

ORMs are excellent for standard CRUD and moderately complex queries, but complex reporting queries, bulk operations across millions of rows, or database-specific features (full-text search, window functions, recursive CTEs) are often better expressed as raw SQL that the ORM executes directly, rather than contorted into whatever subset of SQL the ORM's query builder happens to expose. Nearly every ORM provides an escape hatch for raw queries specifically for this reason, and using it for the specific cases where it's genuinely clearer or more efficient is a sign of good judgment, not a failure to 'use the ORM properly.'

Common Pitfalls

- Accessing a lazily-loaded relation inside a loop over many records, silently reintroducing the N+1 query problem the ORM doesn't prevent by default.
- Never looking at the actual SQL an ORM generates for a non-trivial query, and discovering only in production that it's doing something inefficient.
- Forcing a complex reporting or analytics query through the ORM's query builder instead of dropping to raw SQL where it would be clearer and faster.
- Letting migrations drift between environments — generating a migration locally but forgetting to run it in staging/production, causing schema mismatches.
- Not understanding a cascading delete configured in the ORM's relation definition, and being surprised when deleting a parent silently deletes related child records.

- Fetching and serializing an entire deep object graph (a student with all enrollments, each with all course details) when only a few fields were actually needed.

Key Takeaways

- An ORM's core value is a typed, language-native API over the database plus schema/migration management — not that it eliminates the need to understand SQL.
- Migrations version schema changes reproducibly across environments; auto-generated migrations still deserve a human review before hitting production.
- Lazy loading causes N+1 queries when accessed in a loop; eager loading (include/join) is the standard, explicit fix every mainstream ORM supports.
- Many-to-many relationships needing extra fields on the join itself require modeling the join table explicitly, not relying on an implicit join table.
- Periodically inspect the actual SQL an ORM generates — trusting it blindly is how inefficient queries reach production unnoticed.
- Raw SQL is still the right tool for complex reporting, bulk operations, or database-specific features the ORM's query builder can't cleanly express.

Learn More

Prisma Docs — Getting Started

<https://www.prisma.io/docs/getting-started>

Prisma Docs — Relations

<https://www.prisma.io/docs/orm/prisma-schema/data-model/relation>

Sequelize Docs — Migrations

<https://sequelize.org/docs/v6/other-topics/migrations/>